

DINOv2 Drone Feature Localization Pipeline

Technical Design & Code Migration Documentation

1. Pipeline Architecture Overview

This document describes the optimized, deep learning-based visual feature localization pipeline designed for autonomous drone systems. The system takes a set of reference templates representing target stones or soil features, evaluates camera telemetry from flight stops, and computes high-precision global coordinates using visual scale projection.

To achieve robust matching under motion blur, lighting variations, and classification confusion, the pipeline implements hierarchical noise filtering: Laplacian variance blur estimation, frame-level competitive matching, and stop-wise majority voting.

2. Motion Blur Filtering & Frame Selection

Autonomous drone runs contain motion blur during transition phases. Running template matching on blurred frames corrupts visual texture representations. The pipeline implements a Laplacian variance filter to calculate edge sharpness and selects only the 4 clearest frames per stop group, speeding up downstream execution by 2.5x.

Old Loop (Process All Frames)	New Loop (Process Top 4 Sharpest)
<pre># Loop over all images for count, (img_idx, img_path, yaml_path) in enumerate(query_images, 1): img_bgr = cv2.imread(img_path) # Evaluates every raw frame sequentially coords = load_yaml_coordinates(yaml_path) ...</pre>	<pre># Select 4 sharpest frames using Laplacian Variance stops = {} for (img_idx, img_path, yaml_path) in query_images: group_id = (img_idx // 10) * 10 stops.setdefault(group_id, []).append((img_idx, img_path, yaml_path)) selected_query_images = [] for group_id, group_list in sorted(stops.items()): scored = [] for (img_idx, img_path, yaml_path) in group_list: img = cv2.imread(img_path) gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) var = cv2.Laplacian(gray, cv2.CV_64F).var() scored.append((var, (img_idx, img_path, yaml_path))) scored.sort(key=lambda x: x[0], reverse=True) selected_query_images.extend([x[1] for x in scored[:4]])</pre>

3. Physical Scale Projection & Single-Scale Matching

Instead of searching across multiple query scales which wastes GPU memory and computation, the pipeline uses camera intrinsics (focal length f_y) and frame height z to project query coordinates. The image is scaled exactly so that a 15 cm physical feature matches the 128 px reference template, avoiding scale mismatch and grid alignment errors.

Old Scale Loop (Multi-Scale Search)	New Single-Scale Projection (intrinsics-based)
<pre># Loop over multiple query resolutions for target_h in [336, 448, 560]: target_w = int(round((target_h * (w / h)) / 14)) * 14 query_tensor = preprocess(img, target_w, target_h) # Runs model forward pass multiple times outputs = model(query_tensor) ...</pre>	<pre># Dynamically scale based on physical size and z-height scale = (128.0 * z) / (fy * PHYSICAL_REF_HEIGHT) target_h = max(140, min(560, int(round((orig_h * scale) / 14)) * 14)) target_w = int(round((target_h * (orig_w / orig_h)) / 14)) * 14 # Single forward pass at the exact physical scale resolution query_tensor = preprocess(img_bgr, target_w, target_h) outputs = model(query_tensor)</pre>

4. Single-Pass DINOv2 Feature Extraction

To optimize throughput, query features are extracted exactly once per frame. Similarity convolution is then performed in memory against all reference templates. This completely eliminates redundant forward passes, resulting in a 3x speedup.

Old Pass (Redundant forward run per template)	New Single DINOv2 Pass (Optimal Feature Reuse)
<pre>for ref in refs: # compare_and_localize ran DINOv2 inside loop match_data = compare_and_localize(img, ref)</pre>	<pre># Run DINOv2 exactly once per query image query_tensor = preprocess_image_tensor(img_bgr, target_w, target_h) with torch.no_grad(): outputs = model(pixel_values=query_tensor) query_features = normalize_features(outputs) # Match query features against all templates in-memory for ref in refs: sim_map = compute_dense_similarity(query_features, ref['emb'])</pre>

5. Stop-wise Voting & Coordinate Resolution

Instead of globally averaging matches across the entire flight, the pipeline resolves positions by stop. For each reference template, it counts detections across 10-frame intervals and selects the single 'best stop' (most occurrences, using max confidence as a tie-breaker). Only the coordinates from that group are averaged, minimizing projection errors and false positives.

Old Averaging (Global accumulation)	New Stop-wise Selection (Targeted averaging)
<pre># Global average across all detections feature_coords = {ref['name']: [] for ref in refs} for count, (img, yam1) in enumerate(images): if match_data: feature_coords[ref_name].append(proj_pt)</pre>	<pre># Find best stop group for each unique feature matched_coords = [] for ref in refs: if ref_name in ref_group_counts: # Select group with highest occurrence & max score best_group_id = max(gids, key=lambda g: (count[g], score[g])) best_pts = ref_group_detections[ref_name][best_group_id] pts = np.array([d['x'], d['y'], d['z']] for d in best_pts) mean_pt = np.mean(pts, axis=0) matched_coords.append({ 'ref_name': ref_name, 'x': mean_pt[0], ... })</pre>